Practical - 1

// WRITEA PROGRAM TO IMPLEMENT BUBBLE SORT.

CODE :

```
import java.util.Arrays;

public class CountingSort {
  public CountingSort() {
  }

  public static void main(String[] var0) {
   int[] var1 = new int[]{7, 3, 5, 3, 9, 1};
   int var2 = Arrays.stream(var1).max().getAsInt();
   int[] var3 = new int[var2 + 1];
   int[] var4 = var1;
   int var5 = var1.length;

   for(int var6 = 0; var6 < var5; ++var6) {
     int var7 = var4[var6];
     int var10002 = var3[var7]++;
   }

   int var8 = 0;

   for(var5 = 0; var5 <= var2; ++var5) {
     while(var3[var5]-- > 0) {
      var1[var8++] = var5;
     }
   }

   System.out.println(Arrays.toString(var1));
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac BubbleSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java BubbleSort
[1, 3, 6, 7, 9]
```

Practical - 2

// WRITEA PROGRAM TO IMPLEMENT INSERTION SORT.

CODE :

```
public class InsertionSort {
  public static void insertionSort(int arr[]) {
    for (int i = 1; i < arr.length; i++) {
      int curr = arr[i];
      int prev = i - 1;

      while (prev >= 0 && arr[prev] > curr) {
        arr[prev + 1] = arr[prev];
        prev--;
      }

      arr[prev + 1] = curr;
    }
  }
  public static void printArray(int arr[]) {
    for (int i = 0; i < arr.length; i++) {
      System.out.print(arr[i] + " ");
    }
    System.out.println();
  }

  public static void main(String[] args) {
    int arr[] = {84, 71, 80, 35, 43};

    System.out.print("Unsorted Arrays : ");
    printArray(arr);

    insertionSort(arr);

    System.out.print("Sorted Arrays : ");
    printArray(arr);
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac InsertionSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java InsertionSort
Unsorted Arrays : 84 71 80 35 43
Sorted Arrays : 35 43 71 80 84
```

Practical - 3

// WRITE A PROGRAM TO IMPLEMENT MERGE SORT.

CODE :

```
public class MergeSort {

  // Print Array
  public static void printArr(int arr[]) {
    for (int i = 0; i < arr.length; i++) {
      System.out.print(arr[i] + " ");
    }
    System.out.println();
  }

  // Merge Sort Function
  public static void mergeSort(int arr[], int si, int ei) {
    if (si >= ei) { // Base case
      return;
    }

    int mid = si + (ei - si) / 2;

    mergeSort(arr, si, mid);   // Left part
    mergeSort(arr, mid + 1, ei); // Right part
    merge(arr, si, mid, ei);
  }

  // Merge Function
  public static void merge(int arr[], int si, int mid, int ei) {

    int temp[] = new int[ei - si + 1];

    int i = si;    // left iterator
    int j = mid + 1; // right iterator
    int k = 0;    // temp iterator

    while (i <= mid && j <= ei) {
      if (arr[i] < arr[j]) {
        temp[k++] = arr[i++];
      } else {
        temp[k++] = arr[j++];
      }
    }

    // Left part
    while (i <= mid) {
      temp[k++] = arr[i++];
    }
```

```
    // Right part
    while (j <= ei) {
      temp[k++] = arr[j++];
    }

    // Copy temp to original array
    for (k = 0, i = si; k < temp.length; k++, i++) {
      arr[i] = temp[k];
    }
  }

  public static void main(String[] args) {

    int arr[] = {6, 3, 9, 5, 2, 8};

    System.out.print("Unsorted Array is : ");
    printArr(arr);

    mergeSort(arr, 0, arr.length - 1);

    System.out.print("Sorted Array is : ");
    printArr(arr);
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac MergeSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java MergeSort
Unsorted Array is : 6 3 9 5 2 8
Sorted Array is : 2 3 5 6 8 9
```

Practical - 4

// WRITE A PROGRAM TO IMPLEMENT QUICK SORT.

CODE :

```
import java.util.*;
public class QuickSort {
  static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
    }
  }

  static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
      if (arr[j] < pivot) {
        i++;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
      }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
  }

  public static void main(String[] args) {
    int[] arr = {14, 3, 27, 8, 19};
    quickSort(arr, 0, arr.length - 1);
    System.out.println(Arrays.toString(arr));
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac QuickSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java QuickSort
[3, 8, 14, 19, 27]
```

Practical - 5

// WRITE A PROGRAM TO IMPLEMENT HEAP SORT.

CODE :

```
import java.util.*;
public class HeapSort {
  static void heapify(int[] arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
      largest = left;
    if (right < n && arr[right] > arr[largest])
      largest = right;

    if (largest != i) {
      int temp = arr[i];
      arr[i] = arr[largest];
      arr[largest] = temp;
      heapify(arr, n, largest);
    }
  }
  static void heapSort(int[] arr) {
    int n = arr.length;
    for (int i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
      int temp = arr[0];
      arr[0] = arr[i];
      arr[i] = temp;
      heapify(arr, i, 0);
    }
  }
  public static void main(String[] args) {
    int[] arr = {25, 17, 36, 2, 19};
    heapSort(arr);
    System.out.println(Arrays.toString(arr));
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac HeapSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java HeapSort
[2, 17, 19, 25, 36]
```

Practical - 10

// WRITE A PROGRAM TO IMPLEMENT 0/1 KNAPSACK.

CODE :

```
import java.util.Scanner;
public class KnapsackMatrix {

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int[][] im = new int[10][2];  // weight & value
    int[][] km = new int[11][101]; // DP matrix (adjust size if needed)

    int n, W;
    int A, B;
    System.out.print("\nEnter no. of values = ");
    n = sc.nextInt();

    System.out.print("\nEnter the value of knapsack = ");
    W = sc.nextInt();
    System.out.println("\nEnter the weights and values:");
    for (int i = 0; i < n; i++) {
      im[i][0] = sc.nextInt(); // weight
      im[i][1] = sc.nextInt(); // value
    }

    // Knapsack Matrix
    for (int i = 0; i <= n; i++) {
      for (int j = 0; j <= W; j++) {

        if (i == 0 || j == 0) {
          km[i][j] = 0;
        }
        else if ((j - im[i - 1][0]) < 0) {
          km[i][j] = km[i - 1][j];
        }
        else {
          A = km[i - 1][j];
          B = im[i - 1][1] + km[i - 1][j - im[i - 1][0]];

          if (A > B)
            km[i][j] = A;
          else
            km[i][j] = B;
        }
      }
    }
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac KnapsackMatrix.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java KnapsackMatrix

Enter no. of values = 4

Enter the value of knapsack = 5

Enter the weights and values:
2
12
1
10
3
20
2
15

Knapsack Matrix:
0       0       0       0       0       0
0       0       12      12      12      12
0       10      12      22      22      22
0       10      12      22      30      32
0       10      15      25      30      37
0       10      15      25      30      37

0       10      15      25      30      37

Maximum Profit = 37
0       10      15      25      30      37

0       10      15      25      30      37

Maximum Profit = 37
0       10      15      25      30      37

Maximum Profit = 37
```

Practical - 6

// WRITE A PROGRAM TO IMPLEMENT RADIX SORT.

CODE :

```
import java.util.*;

public class RadixSort {

  static void countSort(int[] arr, int exp) {
    int n = arr.length;
    int[] output = new int[n];
    int[] count = new int[10];

    for (int i = 0; i < n; i++)
      count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
      count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {
      output[count[(arr[i] / exp) % 10] - 1] = arr[i];
      count[(arr[i] / exp) % 10]--;
    }

    System.arraycopy(output, 0, arr, 0, n);
  }

  static void radixSort(int[] arr) {
    int max = Arrays.stream(arr).max().getAsInt();
    for (int exp = 1; max / exp > 0; exp *= 10)
      countSort(arr, exp);
  }

  public static void main(String[] args) {
    int[] arr = {329, 457, 657, 839, 436, 720};
    radixSort(arr);
    System.out.println(Arrays.toString(arr));
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac RadixSort.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> Java RadixSort
[329, 436, 457, 657, 720, 839]
```

Practical - 8

// WRITE A PROGRAM TO IMPLEMENT CHAIN MATRIX MULTIPLICATION.

CODE :

```
public class MatrixChainMultiplication {

  static int matrixChain(int[] p, int n) {
    int[][] m = new int[n][n];

    for (int L = 2; L < n; L++) {
      for (int i = 1; i < n - L + 1; i++) {
        int j = i + L - 1;
        m[i][j] = Integer.MAX_VALUE;

        for (int k = i; k < j; k++) {
          int q = m[i][k] + m[k + 1][j]
              + p[i - 1] * p[k] * p[j];

          if (q < m[i][j])
            m[i][j] = q;
        }
      }
    }
    return m[1][n - 1];
  }

  public static void main(String[] args) {
    int[] arr = {5, 10, 3, 12, 6};
    System.out.println("Minimum Multiplications: " + matrixChain(arr, arr.length));
  }
}
```

OUTPUT :

```
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> javac MatrixChainMultiplication.java
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB> java MatrixChainMultiplication
Minimum Multiplications: 456
PS C:\Users\ASUS\Desktop\Program\Java Programming\DAA LAB>
```